



Advanced Python programming using AI tools

Lecture 1: Intro and Basics

Lecturer: Dr. Havana Rika

Friends or Enemies?



Friends or Enemies?



Gemini Code Assist

Now everyone can get AI-coding assistance from Gemini.

Try at no cost



Listen to article 7 minutes



Our latest developer-oriented [DORA research](#) shows that more than 75% of developers are relying on AI in their daily responsibilities. As one example, more than 25% of all new code at Google is generated by AI, then reviewed and accepted by engineers.

While well-resourced organizations are empowering their engineering teams with the latest AI capabilities, that level of tooling hasn't always been accessible to students, hobbyists, freelancers and startups. And with a worldwide population of developers [forecasted](#) to grow to 57.8 million by 2028, we think AI should be available to them whether they can pay for it or not, so they can start building with what are quickly becoming the standard digital tools of the future.

To bridge that gap, today we're announcing the public preview of [Gemini Code Assist for individuals](#), a free version of our AI-coding assistant.

<https://blog.google/technology/developers/gemini-code-assist-free/>



פגישת צוות בחברת הייטק (צילום: Shutterstock)

כדי שה-AI לא יחליף אתכם – אלה הכישורים הנדרשים עכשיו בהייטק

עידן הבינה המלאכותית משנה את הדרישות מאנשי ההייטק: לצד המיומנויות הטכניות, ההצלחה בתעשייה מחייבת כיום שילוב של הבנה אנושית, כישורי ניהול וחשיבה מערכתית. בדקנו עם בכירים בתעשייה מהם הכישורים שהם מחפשים היום אצל מועמדים לעבודה

הם למדו תואר נחשק, חלמו על משרה בהייטק אבל יושבים בבית בלי עבודה

מספר משרות הג'וניורים החדשות שנפתחו בשנת 2024 נחתך בחצי בהשוואה ל־2023, ובוגרים טריים בלימודי מדעי המחשב יושבים בבית תקופות ארוכות ללא עבודה • כשהבינה המלאכותית כבר יכולה לכתוב קוד ולפתח בעצמה - ברור שזו רק ההתחלה • אילו תפקידים נמצאים בסיכון מוגבר, ומה מפתחי תוכנה יכולים לעשות כדי לתת ערך



23.02.2025

אסף גלעד

<https://www.globes.co.il/news/article.aspx?did=1001502512#>



הם למדו תואר נחשק, חלמו על משרה בהייטק אבל יושבים בבית בלי עבודה

מספר משרות הג'וניורים החדשות שנפתחו בשנת 2024 נחתך בחצי בהשוואה ל־2023, ובוגרים טריים בלימודי מדעי המחשב יושבים בבית תקופות ארוכות ללא עבודה • כשהבינה המלאכותית כבר יכולה לכתוב קוד ולפתח בעצמה - ברור שזו רק ההתחלה • אילו תפקידים נמצאים בסיכון מוגבר, ומה מפתחי תוכנה יכולים לעשות כדי לתת ערך

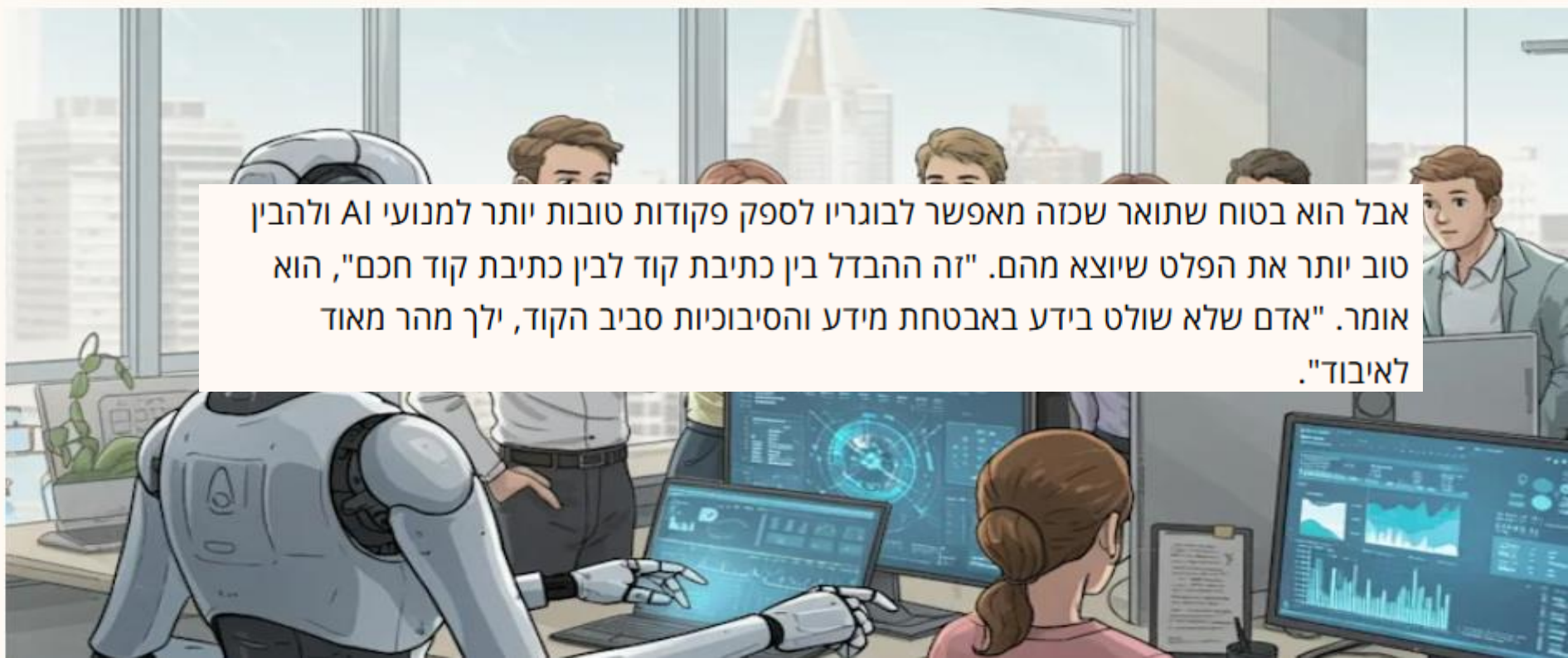


23.02.2025

אסף גלעד

<https://www.globes.co.il/news/article.aspx?did=1001502512#>

אבל הוא בטוח שתואר שכזה מאפשר לבוגריו לספק פקודות טובות יותר למנועי AI ולהבין טוב יותר את הפלט שיוצא מהם. "זה ההבדל בין כתיבת קוד לבין כתיבת קוד חכם", הוא אומר. "אדם שלא שולט בידע באבטחת מידע והסיבוכיות סביב הקוד, ילך מהר מאוד לאיבוד".



הם למדו תואר נחשק, חלמו על משרה בהייטק אבל יושבים בבית בלי עבודה

מספר משרות הג'וניורים החדשות שנפתחו בשנת 2024 נחתך בחצי בהשוואה ל־2023, ובוגרים טריים בלימודי מדעי המחשב יושבים בבית תקופות ארוכות ללא עבודה • כשהבינה המלאכותית כבר יכולה לכתוב קוד ולפתח בעצמה - ברור שזו רק ההתחלה • אילו תפקידים נמצאים בסיכון מוגבר, ומה מפתחי תוכנה יכולים לעשות כדי לתת ערך

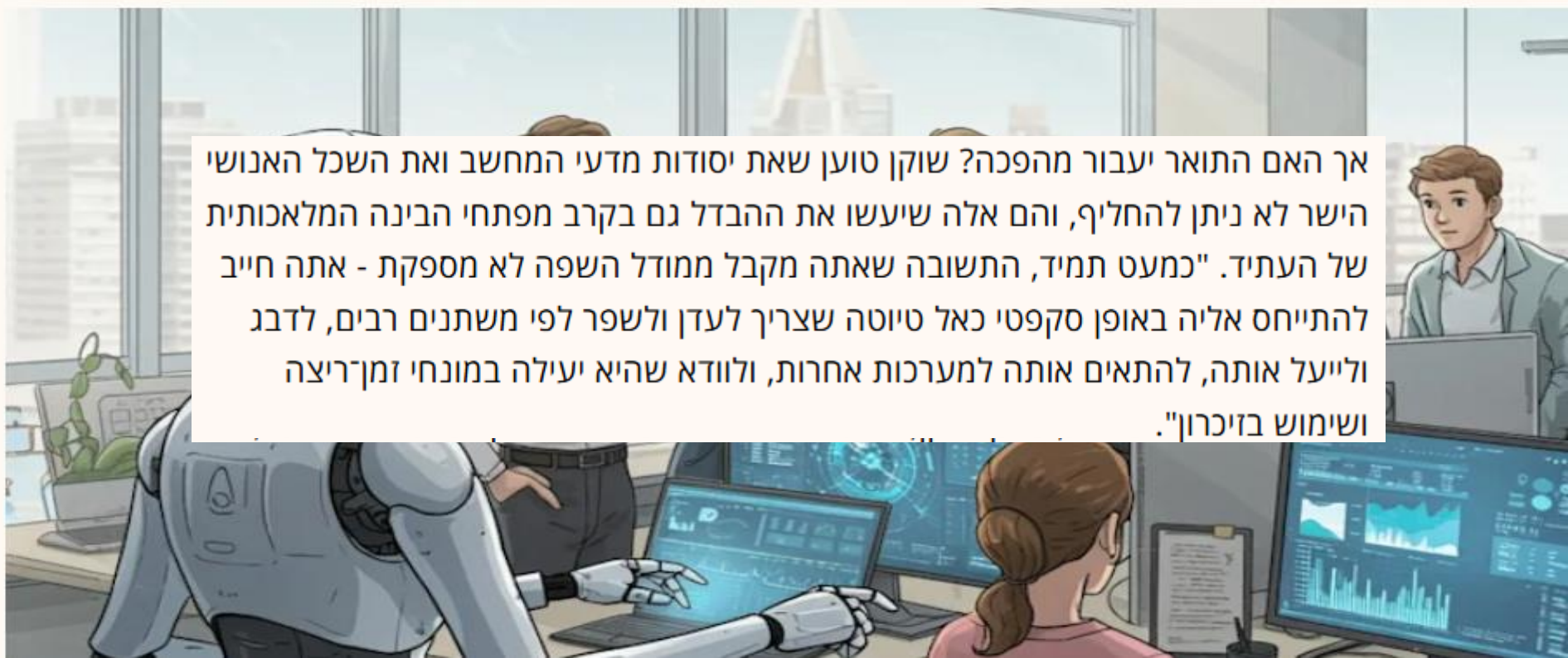


23.02.2025

אסף גלעד

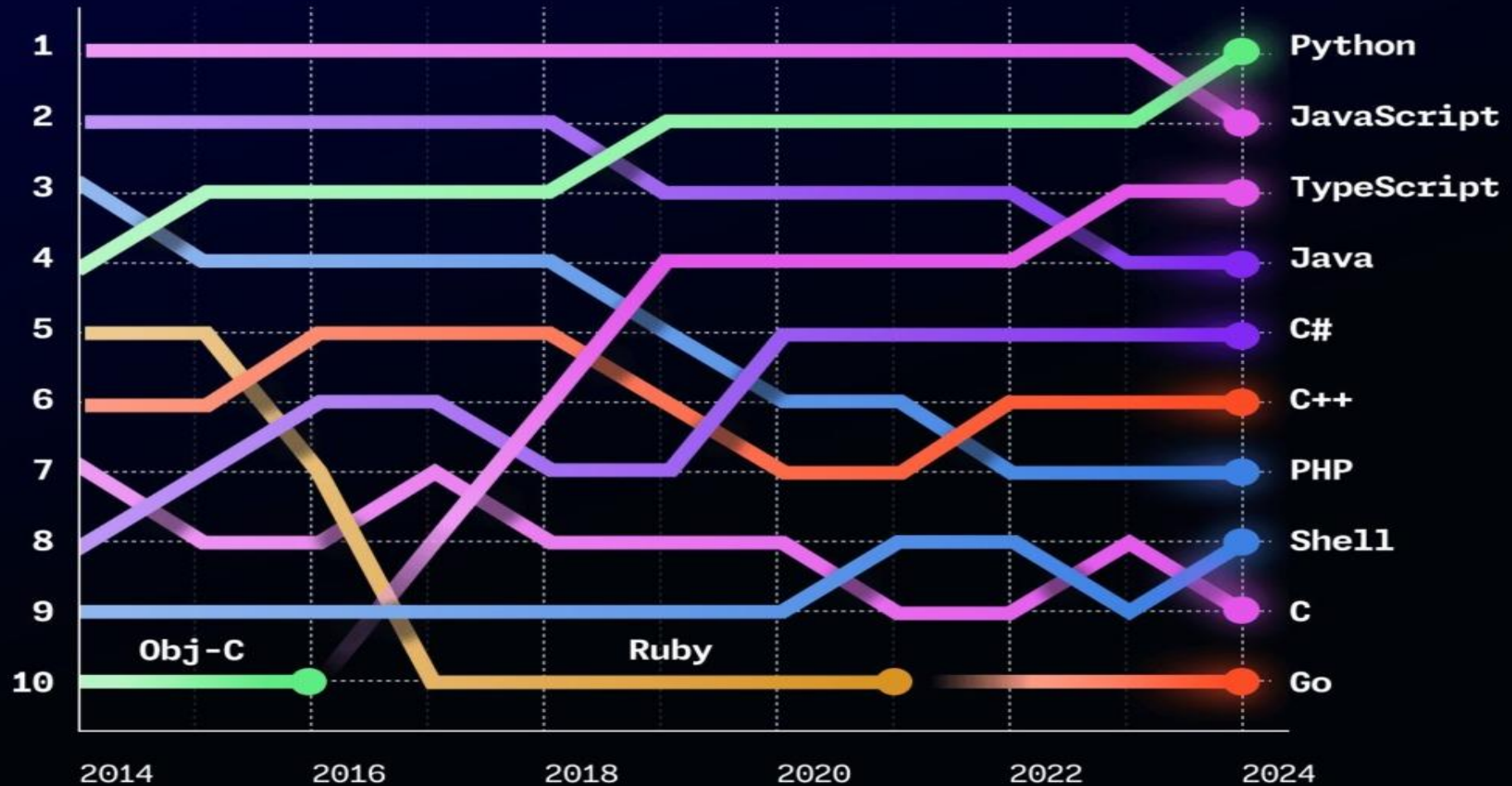
<https://www.globes.co.il/news/article.aspx?did=1001502512#>

אך האם התואר יעבור מהפכה? שוקן טוען שאת יסודות מדעי המחשב ואת השכל האנושי הישר לא ניתן להחליף, והם אלה שיעשו את ההבדל גם בקרב מפתחי הבינה המלאכותית של העתיד. "כמעט תמיד, התשובה שאתה מקבל ממודל השפה לא מספקת - אתה חייב להתייחס אליה באופן סקפטי כאל טיוטה שצריך לעדן ולשפר לפי משתנים רבים, לדבג ולייצל אותה, להתאים אותה למערכות אחרות, ולוודא שהיא יעילה במונחי זמן־ריצה ושימוש בזיכרון".

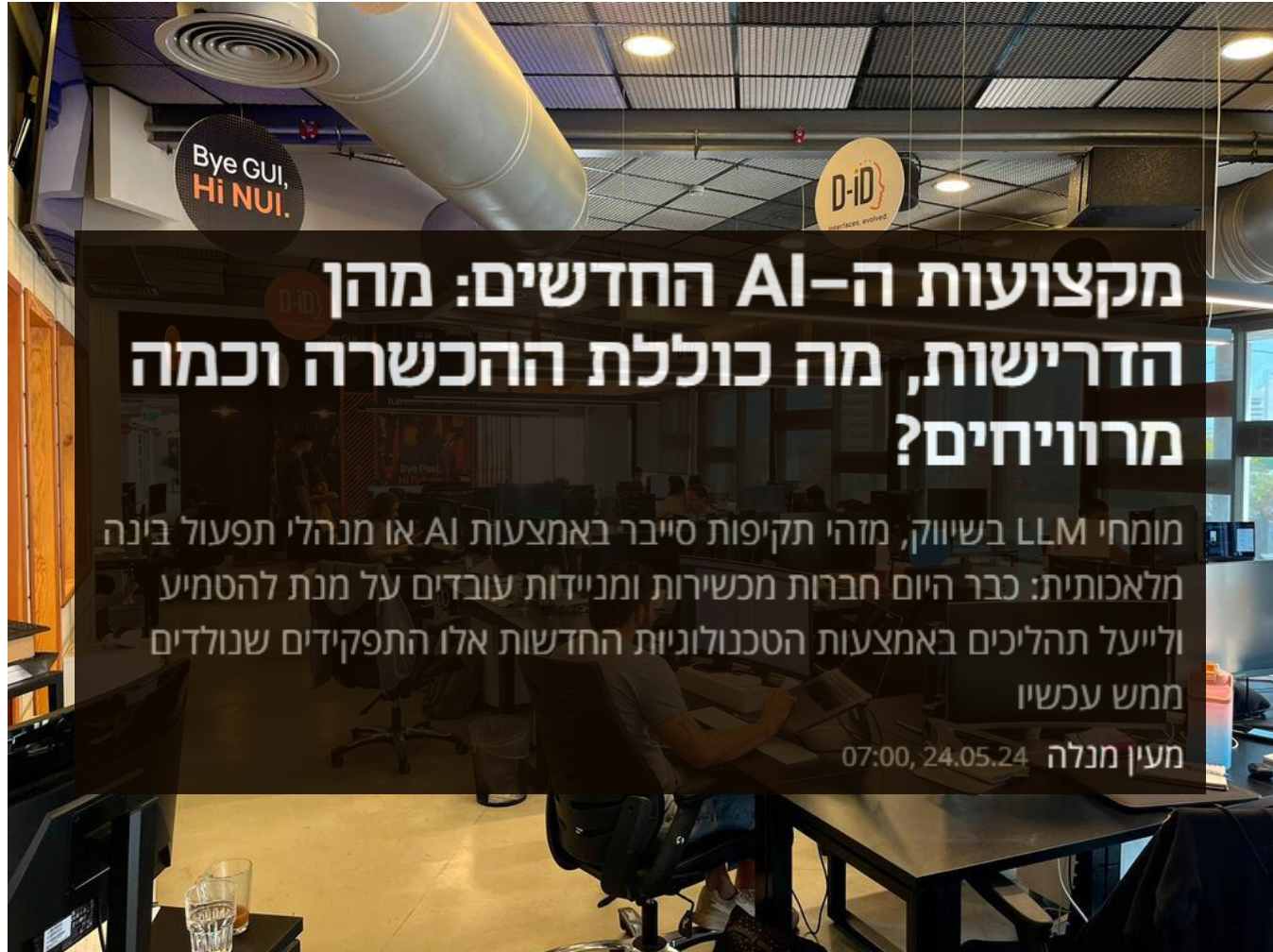


Top programming languages on GitHub

RANKED BY COUNT OF DISTINCT USERS CONTRIBUTING TO PROJECTS OF EACH LANGUAGE.



Self - Learning



Self - Learning



דרישות התפקיד: ידע מוקדם בכלי GenAI מתקדמים. כגון ChatGPT ו-ComfyUI. הכישורים האישיותיים והלימודיים כוללים יכולת ליזום ויכולת למידה עצמית.

מה שחשוב הוא ראייה מאד ביקורתית והבנה מערכתית של איך דברים עובדים, ומקרנות רבה לגבי העולם המתפתח של generative ai. חלק גדול מהתפקיד: כרוך בהתעדכנות שוטפת של ההתפתחויות בתחום, ולהבין מה רלוונטי לשילוב בתהליכים שרצים היום", הוא אומר.

דרישות התפקיד: יכולת ליצירה וניהול שיתוף פעולה חלק בין המחלקות, כדי ליעל את תהליך אספקת המוצר לשוק והשגת שביעות רצון מצד הלקוחות. נחוצה יכולת בניהול פרויקטים טכניים, כישורי תקשורת וכישורים ארגוניים חזקים, וכדי להצטיין בתפקיד צרי להיות בעלי יכולת הסתגלות, לפעול באופן אנליטי ולפעול יצירתית לפתרון בעיות.

דרישות התפקיד: חוקרים ואנליסטים מתחום הסייבר עם ניסיון טכנולוגי, רצוי בעלי היכרות עם תחומי הדאטה וה-AI.

דרישות התפקיד: הכרת כלים טכנולוגיים, יכולת למידה עצמאית, ירידה לפרטים הקטנים, יכולת מחקר ותשוקה ללמידת השינויים הטכנולוגיים המהירים שמתרחשים בתחום ה-AI.

דרישות התפקיד: הבנה בשיווק, היכרות וניסיון בכלי GenAI מתקדמים בדגש על עולמות הוידאו והתוכן, יכולת למידה עצמית.

דרישות התפקיד: "אנו מחפשים ניסיון והבנה מעמיקה של כלי בינה מלאכותית ולמידת מכונה. כמו גם כישורי תכנות בשפות כמו Python, יכולת אנליטית ופתרון בעיות טכניות מורכבות", אומר בן חיים.

Grading

- **10% Notebooks.**
 - Binary grade.
 - Submission in individuals.

- **30% 3 Projects.**
 - Numeric grade.
 - Submission in pairs.

- **60% final exam.**

Contact Details

- Dr. Havana Rika
 - Email: havanari@mta.ac.il
 - Reception hours: by appointment

Today

- ❖ Intro
- ❖ Variables
- ❖ Math operations
- ❖ Casting
- ❖ Input/Output
- ❖ If-else

Intro to Python

- **Interpreted (script) high-level** programming language.
- Emphasizes code **readability**.
- Aim to help programmers write **clear, logical** code.
- **Guido Van Rossum** designed **Python** and released the first version in **1991**.
 - Python 2.0 was released in 2000
 - Python 3.0 was released in 2008
- We will support version 3.8.3.



Why Python?

- Python works on **different platforms**.
- Python has a simple syntax **similar to the English language**.
- Python has syntax that allows developers to write **programs** with **fewer lines** than some other programming languages.
- Python runs on an **interpreter** system, meaning that code can be executed as soon as it is written.
- Python can be treated in a **procedural** way or an **object-oriented** way.



Python modules

The Python Package Index (PyPI) contains over 530,000 packages with a wide range of functionality, including:

- Automation
- Data analytics
- Databases
- Documentation
- Graphical user interfaces
- Image processing
- Machine learning
- Mobile apps
- Multimedia
- Computer networking
- Scientific computing
- System administration
- Test frameworks
- Text processing
- Web frameworks
- Web scraping



Writing Code

- Python syntax can be executed by writing directly in the Command Line.

```
>>> print("Hello, World!")  
Hello, World!
```

- Or by creating a python file on the server, using the .py file extension.

```
C:\Users\Your Name>python myfile.py
```

- You can use the PyCharm editor for creating and running .py files.



Comments & Indentation

- Line **comments** starts with a #.

```
#This is a comment  
print("Hello, World!")
```

```
print("Hello, World!") #This is a comment
```

```
#This is a comment  
#written in  
#more than just one line  
print("Hello, World!")
```

- Python uses **indentation** to indicate a **block of code**.

```
if 5 > 2:  
    print("The term is true.")  
    print("Five is greater than two!")  
else:  
    print("The term is false.")  
print("Bye")
```

Variables

- Python has no command for declaring a variable.
- A **variable** is **created** the moment you **first assign** a **value** to it.

```
x = 5  
y = "John"  
print(x)  
print(y)
```

- Variables do **not need to be declared with any particular type**, and can even **change type** after they have been set.

```
x = 4          # x is of type int  
x = "Sally"    # x is now of type str  
print(x)
```


Var Types

Type	Example
String	'hi' OR "hi"
int	-786
Float	-21.9
Complex	9.322e-36j
List	[1,12,15,13]
Set	{1,2,3}
Tuple	(3,11,67)
Dictionary	{a:6,b:8,c:4,d:3}

Casting

- If you want to **specify the data type** of a variable, this can be done with casting.

```
x = str(3)    # x will be '3'  
y = int(3)    # y will be 3  
z = float(3)  # z will be 3.0
```

- You can **get the data type** of a variable with the **type()** function.

```
x = 5  
y = "John"  
print(type(x))  
print(type(y))
```

Arithmetic Operators

Operator	Name	Example
+	Addition	$5+2=7$
-	Subtraction	$5-2=3$
*	Multiplication	$5*2=10$
/	Division	$5/2=2.5$
%	Modulus	$5\%2=1$
**	Exponentiation	$5**2=25$
//	Floor division	$5//2=2$

Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

Print

- Syntax: **print(arg0, arg1, arg2,...)**
- print 1 space between args.
- print “\n” at the end of the line.
- Example:
 print('a', 1 , 2, “b”)
 print('summer time')

```
a 1 2 b
summer time
```

Print Sep

- Optional.
- Specify how to separate the objects, if there is more than one.
- Default is sep=" ".

`print(arg0, arg1,..., sep="<some symbols>")`

```
print("a", "b", "c")  
print(1,2,3)
```

```
a b c  
1 2 3
```

```
print("a", "b", "c", sep="##")  
print(1,2,3)
```

```
a##b##c  
1 2 3
```

```
print("a", "b", "c", sep="@")  
print(1,2,3, sep="---")
```

```
a@b@c  
1---2---3
```

Print end

- Optional.
- Specify what to print at the end.
- Default is end='\n'.

`print(arg0, arg1,..., end="<some symbols>")`

```
print("a",1, end="!")  
print("b","c")
```

a 1!b c

```
print("a",1, end="!\n")  
print("b","c")
```

a 1!
b c

```
print("a",1, end="!\n")  
print("b","c","f", sep="~", end="\n%%%\n")  
print("d", "e")
```

a 1!
b~c~f
%%%
d e

Input

- Python allows for user input.
- That means we are able to ask the user for input.
- Syntax: `varName = input(message)`

• Example:

```
username = input("Enter username:")  
print("Username is: " + username)
```

```
Enter username: Tal  
Username is: Tal
```

- `input()` return String type.
- We can cast the input to our requested type:

```
age=int(input("enter age"))  
salary=float(input("enter salary"))
```

Python Booleans

- Booleans represent one of two values: True OR False.
- In math operations True is 1 and False is 0.

```
print(1+True)
print(5*False)
```

```
2
0
```

```
print(10 > 9)
print(10 == 9)
print(10 < 9)
```

- Convert values by **bool()**.
- Almost all values cast to True except 0 or "" which are False.

```
print(bool("Hello"))
print(bool(15))
```


If – elif - else

```
age = int(input('Enter age:\n'))  
  
if age<18:  
    print('teenager or child')  
elif 18 <= age < 30:  
    print('young adult')  
else:  
    print ('adult')
```

```
x = 41  
if x > 10:  
    print("Above ten, ")  
    if x > 20:  
        print("and also above 20!")  
    else:  
        print("but not above 20. ")
```

Comparison Operators

Operator	Name	Example
==	Equal	<code>x == y</code>
!=	Not equal	<code>x != y</code>
>	Greater than	<code>x > y</code>
<	Less than	<code>x < y</code>
>=	Greater than or equal to	<code>x >= y</code>
<=	Less than or equal to	<code>x <= y</code>

Logical Operators

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverse the result, returns False if the result is true	<code>not(x < 5 and x < 10)</code>